C-DAC Mumbai Day 06- DBT

1. What is an index?

An index is a data structure that helps MySQL find rows without checking every row in a table. Think of the index at the back of a textbook: it points you to the pages where a topic appears. A database index stores indexed column values along with information that helps locate the matching row.

Without a useful index, MySQL may perform a full table scan. With a useful index, it may read a much smaller part of the index and fetch matching rows. The optimizer decides whether an index is beneficial for a particular query.

```
mysql> show index from employee;
+----------+------------+------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---
-----+---------------+---------+------------+
| Table    | Non_unique | Key_name               | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Com
ment | Index_comment | Visible | Expression |
+----------+------------+------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---
-----+---------------+---------+------------+
| employee |          0 | PRIMARY                |            1 | emp_id      | A         |          10 |     NULL |   NULL |      | BTREE      |
     |               | YES     | NULL       |
| employee |          1 | fk_department_dept_id |            1 | dept_id     | A         |           4 |     NULL |   NULL | YES  | BTREE      |
     |               | YES     | NULL       |
+----------+------------+------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---
-----+---------------+---------+------------+
2 rows in set (0.06 sec)
```

2. B-tree idea in brief

Many InnoDB indexes use a B-tree variant called a B+ tree. Values are kept in sorted order. Internal tree pages guide the search toward leaf pages; leaf pages contain index entries. This structure supports equality lookups, range searches and ordered retrieval efficiently.

Example: WHERE emp_id = 25 can navigate to the matching key. WHERE salary BETWEEN 50000 AND 70000 can scan a range of sorted index entries. Exact implementation details are internal to MySQL and may vary by index type.

3. Benefits and costs

Benefits	Costs / trade-offs
Faster filtering, joins, and lookups when the index matches the query	Extra disk space for index pages
Can help ORDER BY and GROUP BY in suitable cases	INSERT, UPDATE and DELETE must maintain affected indexes
A UNIQUE index enforces uniqueness	Too many or poorly chosen indexes can slow writes and increase maintenance
May reduce rows examined	An index may not help if a large share of the table is needed

Good practice: index columns used often in WHERE, JOIN, and sometimes ORDER BY/GROUP BY, but choose based on actual queries and verify with EXPLAIN. Do not add an index to every column automatically.

4. Types of indexes

4.1 Primary key index

A PRIMARY KEY uniquely identifies each row and does not allow NULL. In InnoDB, the primary key is the clustered index: table rows are organized by primary-key key values in the clustered index.

```
CREATE TABLE employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    dept_id INT,
    salary DECIMAL(10,2)
);
```

This means the table's actual data is organized and stored in the structure of the primary key's index.

A clustered index is an index that determines how the actual table records are organized in the database's storage structure.

```
mysql> create index idx_emp_emaild_uq
    -> on emp_demo(emailid);
Query OK, 0 rows affected (0.10 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> show index from emp_demo;
+----------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------
--+---------------+---------+------------+
| Table    | Non_unique | Key_name         | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment
 | Index_comment | Visible | Expression |
+----------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------
--+---------------+---------+------------+
| emp_demo |          0 | PRIMARY          |            1 | emp_id      | A         |           0 |     NULL |   NULL |      | BTREE      |
 |               | YES     | NULL       |
| emp_demo |          1 | idx_emp_emaild_uq |            1 | emailid     | A         |           0 |     NULL |   NULL | YES  | BTREE      |
 |               | YES     | NULL       |
+----------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------
--+---------------+---------+------------+
2 rows in set (0.00 sec)
```

Can a table have multiple clustered indexes?

No. An InnoDB table can have only one clustered index because its complete row data is stored in that index structure.

4.2 Unique index

A UNIQUE index prevents duplicate indexed key values. In MySQL, a UNIQUE index can allow multiple NULL values for a nullable column, because NULL values are treated specially for uniqueness.

```
CREATE UNIQUE INDEX uq_employee_email
ON employee(email);
```

The example assumes employee has an email column. A UNIQUE constraint can also be declared when creating or altering a table.

```
mysql> create index idx_emp_emaild_uq
    -> on emp_demo(emailid);
Query OK, 0 rows affected (0.10 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> show index from emp_demo;
+----------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------
--+---------------+---------+------------+
| Table    | Non_unique | Key_name         | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment
 | Index_comment | Visible | Expression |
+----------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------
--+---------------+---------+------------+
| emp_demo |          0 | PRIMARY          |            1 | emp_id      | A         |           0 |     NULL |   NULL |      | BTREE      |
 |               | YES     | NULL       |
| emp_demo |          1 | idx_emp_emaild_uq |            1 | emailid     | A         |           0 |     NULL |   NULL | YES  | BTREE      |
 |               | YES     | NULL       |
+----------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------
--+---------------+---------+------------+
2 rows in set (0.00 sec)
```

4.3 Single-column index

```
CREATE INDEX idx_employee_salary
ON employee(salary);
```

This index may help queries that filter or sort by salary, depending on the query and data distribution.

4.4 Composite (multi-column) index

```
CREATE INDEX idx_dept_salary
ON employee(dept_id, salary);
```

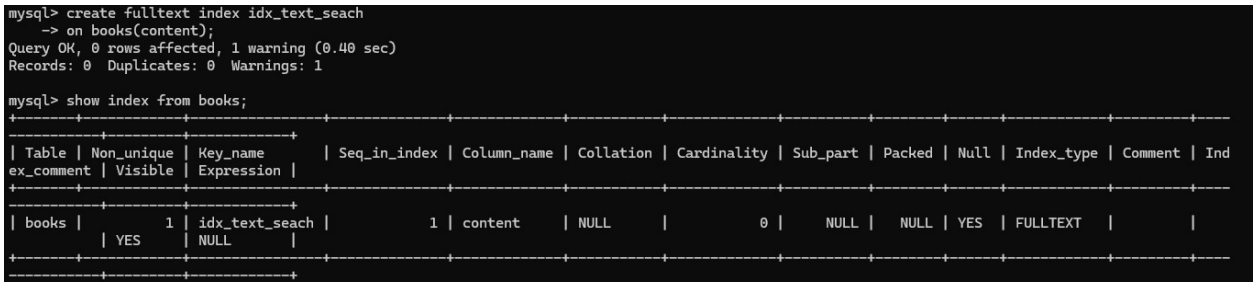
This index stores keys ordered first by dept_id, then by salary within each dept_id. The column order is important.

```
mysql> create index idx_emp_id_salary
    -> on emp_demo(emp_id, salary);
Query OK, 0 rows affected (0.07 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> show index from emp_demo;
+----------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------
--+---------------+---------+------------+
| Table    | Non_unique | Key_name         | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment
 | Index_comment | Visible | Expression |
+----------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------
--+---------------+---------+------------+
| emp_demo |          0 | PRIMARY          |            1 | emp_id      | A         |           0 |     NULL |   NULL |      | BTREE      |
 |               | YES     | NULL       |
| emp_demo |          1 | idx_emp_emaild_uq |            1 | emailid     | A         |           0 |     NULL |   NULL | YES  | BTREE      |
 |               | YES     | NULL       |
| emp_demo |          1 | idx_emp_salary   |            1 | salary      | A         |           0 |     NULL |   NULL | YES  | BTREE      |
 |               | YES     | NULL       |
| emp_demo |          1 | idx_emp_id_salary |            1 | emp_id      | A         |           0 |     NULL |   NULL |      | BTREE      |
 |               | YES     | NULL       |
| emp_demo |          1 | idx_emp_id_salary |            2 | salary      | A         |           0 |     NULL |   NULL | YES  | BTREE      |
 |               | YES     | NULL       |
+----------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------
--+---------------+---------+------------+
5 rows in set (0.01 sec)
```

4.5 FULLTEXT index

A FULLTEXT index supports text searching for words or terms in supported character columns, rather than ordinary exact-match or prefix lookup. MySQL supports FULLTEXT with InnoDB and MyISAM for supported text columns.



```
mysql> create fulltext index idx_text_seach
    -> on books(content);
Query OK, 0 rows affected, 1 warning (0.40 sec)
Records: 0  Duplicates: 0  Warnings: 1

mysql> show index from books;
+-------+------------+---------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+----
-----------+---------+------------+
| Table | Non_unique | Key_name      | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Ind
ex_comment | Visible | Expression |
+-------+------------+---------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+----
-----------+---------+------------+
| books |          1 | idx_text_seach |            1 | content     | NULL      |           0 |     NULL |   NULL | YES  | FULLTEXT   |         |
          | YES     | NULL       |
+-------+------------+---------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+----
-----------+---------+------------+
```

5. Clustered vs secondary indexes in InnoDB

Clustered index	Secondary index
The leaf pages contain the row data itself.	The leaf pages contain indexed columns plus the primary-key value used to locate the row.
An InnoDB table has one clustered index, usually the PRIMARY KEY.	A table can have multiple secondary indexes.
If no primary key exists, InnoDB selects a suitable NOT NULL unique index; otherwise it creates a hidden clustered row ID.	A secondary-index lookup may require an additional lookup in the clustered index to fetch other columns.

This explains why short, stable primary keys are often useful: the primary-key value is carried in secondary-index entries.

6. CREATE INDEX, DROP INDEX and SHOW INDEX

```
CREATE INDEX idx_employee_dept
ON employee(dept_id);

DROP INDEX idx_employee_dept
ON employee;

SHOW INDEX FROM employee;
```

CREATE INDEX adds an index. DROP INDEX removes an index but does not delete table rows. SHOW INDEX displays index metadata, including index name, uniqueness, indexed columns and their order.

Primary-key indexes are normally managed with ALTER TABLE ... DROP PRIMARY KEY rather than DROP INDEX.

7. EXPLAIN basics

EXPLAIN shows MySQL's planned approach for executing a SELECT query. It helps you see whether MySQL expects to use an index and how it plans to access tables. EXPLAIN describes a plan; it does not by itself prove actual runtime performance.

```
EXPLAIN
SELECT emp_id, emp_name
FROM employee
WHERE dept_id = 10;
```

EXPLAIN field	Basic meaning
type	Access method, such as const, ref, range, index or ALL; ALL commonly means a full table scan.
possible_keys	Indexes MySQL considers potentially usable.
key	Index MySQL selected, if any.
rows	Estimated number of rows examined.
Extra	Additional information, such as Using where or Using index.

The best plan depends on table size, statistics, selectivity, and the query. A full table scan is not automatically bad for a small table or a query that needs most rows.

8. Leftmost prefix rule

For an index on (dept_id, salary, emp_name), MySQL can generally use the leftmost leading parts: dept_id; (dept_id, salary); or (dept_id, salary, emp_name). An index starting with dept_id is not generally a direct lookup index for salary alone.

```
CREATE INDEX idx_dept_salary_name
ON employee(dept_id, salary, emp_name);

-- Can use the leading dept_id part
SELECT * FROM employee WHERE dept_id = 10;

-- Can use both leading columns
SELECT * FROM employee
WHERE dept_id = 10 AND salary > 60000;

-- Usually cannot use this composite index for a direct salary-only lookup
SELECT * FROM employee WHERE salary > 60000;
```

This is a general rule, not a promise about every execution plan. Optimizer features and query shape can affect index use. Choose column order based on common filters, joins, sorting, and selectivity.

```
mysql> show tables;
+------------------------------+
| Tables_in_oct13              |
+------------------------------+
| average_salary_per_dept      |
| department                   |
| employee                     |
| sales_employee_with_dept_name |
| sales_employees              |
| select_employee              |
+------------------------------+
6 rows in set (0.01 sec)

mysql> show full tables;
+------------------------------+------------+
| Tables_in_oct13              | Table_type |
+------------------------------+------------+
| average_salary_per_dept      | VIEW       |
| department                   | BASE TABLE |
| employee                     | BASE TABLE |
| sales_employee_with_dept_name | VIEW       |
| sales_employees              | VIEW       |
| select_employee              | VIEW       |
+------------------------------+------------+
6 rows in set (0.01 sec)
```

```
mysql> create table employee_demo ( emp_id int primary key,
    ->  emp_name varchar (50),
    ->  salary decimal (10,2),
    ->  dept_id int );
Query OK, 0 rows affected (0.04 sec)
```

```
mysql>  delimiter //
mysql>  create procedure insert_into_employee()
    ->  begin
    ->
    ->  declare i Int default 1;
    ->  while i <= 100000 do
    ->  insert into employee_demo
    ->  values (
    ->  i,
    ->  concat('employee_', i),
    ->  30000+ (i mod 70000),
    ->  (i mod 10) +1
    ->  );
    ->  set i=i+1;
    ->  end while;
    ->  end//
Query OK, 0 rows affected (0.01 sec)

mysql>  delimiter ;
mysql> call insert_into_employee();
```

```
mysql> show procedure status
    -> where db='oct13';
+-------+----------------------+-----------+----------------+---------------------+---------------------+---------------+---------+----------------
-----+----------------------+--------------------+
| Db    | Name                 | Type      | Definer        | Modified            | Created             | Security_type | Comment | character_set_cl
ient | collation_connection | Database Collation |
+-------+----------------------+-----------+----------------+---------------------+---------------------+---------------+---------+----------------
-----+----------------------+--------------------+
| oct13 | get_all_students     | PROCEDURE | root@localhost | 2026-10-14 07:39:55 | 2026-10-14 07:39:55 | DEFINER       |         | cp850
     | cp850_general_ci     | utf8mb4_0900_ai_ci |
| oct13 | insert_into_employee | PROCEDURE | root@localhost | 2026-10-14 10:53:21 | 2026-10-14 10:53:21 | DEFINER       |         | cp850
     | cp850_general_ci     | utf8mb4_0900_ai_ci |
| oct13 | show_student         | PROCEDURE | root@localhost | 2026-10-14 11:57:22 | 2026-10-14 11:57:22 | DEFINER       |         | cp850
     | cp850_general_ci     | utf8mb4_0900_ai_ci |
+-------+----------------------+-----------+----------------+---------------------+---------------------+---------------+---------+----------------
-----+----------------------+--------------------+
3 rows in set (0.00 sec)
```

Before index

-> Filter: (employee_demo.emp_name = 'employee_99587') (cost=10081.95 rows=10002) (actual time=44.195..44.330 rows=1 loops=1)

-> Table scan on employee_demo (cost=10081.95 rows=100017) (actual time=0.059..28.621 rows=100000 loops=1)

After index

**+----------+----------+----------

3. Why Do We Need Stored Programs?

Imagine a college registration system. Whenever a student registers, the application may need to insert the student details, check the department, update related information and display the registration details.

The application could send several SQL statements separately. If the same database logic is needed in many places, the SQL may be repeated. A stored procedure allows related SQL statements to be stored under one name in the database and called when required.

Example: Without and with a procedure

```
INSERT INTO student (student_id, name, dept_id)
VALUES (101, 'Amit', 10);

SELECT *
FROM student
WHERE student_id = 101;
```

The statements above can be grouped into a stored procedure. A procedure does not run merely because it is created; it runs when called.

```
DELIMITER //

CREATE PROCEDURE get_student()
BEGIN
    SELECT *
    FROM student
    WHERE student_id = 101;
END //

DELIMITER ;

CALL get_student();
```

When are procedures useful?

- When the same database operation is performed repeatedly.
- When several SQL statements form one database-side task.
- When business rules should be maintained centrally.
- When database access should be controlled through permissions.
- When an application can benefit from sending one procedure call instead of many individual statements.

4. Types of Stored Programs in MySQL

Type	Purpose	How it runs
Stored procedure	Performs a task or a group of SQL operations.	Called explicitly using CALL.
Stored function	Calculates and returns a value.	Used in an expression or SQL statement.

Trigger	Responds automatically to a table event.	Fires for a configured INSERT, UPDATE or DELETE event.
Event	Runs a scheduled database task.	Runs according to its schedule when the event scheduler is enabled.

Examples:

- Procedure: retrieve students belonging to a department.
- Function: calculate a student's percentage and return it.
- Trigger: write an audit record when a student's department changes.
- Event: perform a scheduled cleanup of expired temporary records.

A procedure can return query result sets and can also use OUT or INOUT parameters. A function returns a value through its function result. These are different ways of providing output.

5. Benefits and Drawbacks

Benefits

- Reusability: the same procedure can be called repeatedly.
- Reduced network traffic: one call can replace several separate client-server requests.
- Centralized logic: rules can be maintained in one place for multiple applications.
- Security: permissions can allow users to execute specific procedures without granting broad direct table access; behavior depends on security context and configuration.
- Maintainability: shared database logic does not need to be duplicated across applications.
- Potential performance benefits: stored routines can avoid repeatedly sending and parsing the routine definition, but they are not automatically faster than equivalent SQL.

Drawbacks

- Procedures require knowledge of MySQL procedural SQL.
- Debugging may be less convenient than debugging application code.
- Complex database-side logic can be difficult to maintain.
- Procedures can be database-specific and make migration harder.
- Poorly written procedures can still cause slow queries, locking or excessive resource use.
- Splitting business logic between an application and the database can make the system harder to understand.

6. DELIMITER: What and Why?

A delimiter is the character or sequence used by a client to identify the end of a statement. The usual delimiter in MySQL clients is a semicolon (;).

A procedure may contain several semicolons inside its BEGIN ... END block. If the client treats the first semicolon as the end of the whole definition, the procedure definition may be sent too early. Changing the delimiter allows the internal semicolons to remain part of the procedure definition.

```
DELIMITER //

CREATE PROCEDURE show_students()
BEGIN
    SELECT * FROM student;
    SELECT COUNT(*) FROM student;
END //

DELIMITER ;
```

DELIMITER // changes the client-side statement terminator to //.
Semicolons inside BEGIN ... END terminate statements within the procedure.
END // marks the end of the complete procedure definition for the client.
DELIMITER ; restores the usual delimiter.

Important: DELIMITER is a command understood by clients such as the MySQL command-line client and Workbench SQL editor. It is not an SQL statement sent to the server. The delimiter // is a convention, not a requirement; $$ can also be used.

7. Prepare the Example Database

Use the following setup if the classroom database does not already contain these tables. If you have already created them, do not run the CREATE statements again.

```
CREATE DATABASE college_db;
USE college_db;

CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);

CREATE TABLE student (
    student_id INT PRIMARY KEY,
    name VARCHAR(40) NOT NULL,
    dept_id INT NULL,
    marks DECIMAL(5,2),
    CONSTRAINT fk_student_department
        FOREIGN KEY (dept_id)
        REFERENCES department(dept_id)
);

INSERT INTO department VALUES
(10, 'Computer Science'),
(20, 'Information Technology'),
(30, 'Electronics');

INSERT INTO student VALUES
(101, 'Amit', 10, 85.50),
(102, 'Neha', 20, 92.00),
```

```
(103, 'Rahul', 10, 76.50),
(104, 'Priya', 30, 88.00);
```

8. Create and Execute a First Procedure

Procedure without parameters

```
DELIMITER //

CREATE PROCEDURE get_all_students()
BEGIN
    SELECT * FROM student;
END //

DELIMITER ;

CALL get_all_students();
```

CREATE PROCEDURE stores the procedure definition. CALL executes it. The SELECT inside the procedure returns the student rows as a result set.

Part	Meaning
CREATE PROCEDURE	Creates a stored procedure.
get_all_students()	Procedure name; this example has no parameters.
BEGIN	Starts the compound statement block.
SELECT * FROM student;	SQL statement executed by the procedure.
END	Ends the block.
CALL	Executes the procedure.

BEGIN ... END

BEGIN ... END groups multiple statements into one compound statement block.

```
DELIMITER //

CREATE PROCEDURE student_report()
BEGIN
    SELECT * FROM student;
    SELECT COUNT(*) AS total_students FROM student;
END //

DELIMITER ;

CALL student_report();
```

This call produces two result sets: the student records and the total student count. BEGIN ... END does not, by itself, start a transaction or make all statements automatically reversible.

9. Managing Stored Procedures

Execute a procedure

```
CALL procedure_name();

CALL get_all_students();
```

Drop a procedure

```
DROP PROCEDURE procedure_name;

DROP PROCEDURE IF EXISTS get_all_students;
```

Dropping a procedure removes its definition, not the table or records used by the procedure.

Show procedure status

```
SHOW PROCEDURE STATUS;

SHOW PROCEDURE STATUS
WHERE Db = 'databasename';
```

This displays metadata about procedures visible to the current account, including the database, name and creation details.

Show the procedure definition

```
SHOW CREATE PROCEDURE procedure_name;
```

Use this command to inspect the stored procedure definition.

Command	Purpose
CREATE PROCEDURE	Create a procedure.
CALL	Execute a procedure.
DROP PROCEDURE	Remove a procedure.
SHOW PROCEDURE STATUS	List procedure metadata.
SHOW CREATE PROCEDURE	Inspect a procedure definition.

10. Variables in Stored Procedures

Variables hold intermediate values that can be used in calculations, conditions or queries.

Local variables: DECLARE

```
DECLARE total_students INT DEFAULT 0;
DECLARE student_name VARCHAR(40);
DECLARE student_marks DECIMAL(5,2);
```

- Declare local variables at the beginning of the relevant BEGIN ... END block, before executable statements in that block.
- Choose a data type appropriate for the value.

- A local variable is available within its declaring block and nested blocks.
- A local variable does not use the @ prefix.

Assign values with SET

```
DELIMITER //

CREATE PROCEDURE calculate_total1()
BEGIN
    DECLARE total INT DEFAULT 0;

    SET total = 100+ 200;

    SELECT total AS total_value;
END //

DELIMITER ;

CALL calculate_total();
```

```
mysql>  delimiter //
mysql>  create procedure calculate_total1()
    ->  begin
    ->  declare total int default 0;
    ->  set total = 100+200;
    ->  select total as total_value;
    ->  end //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql>  delimiter ;
```

Expected output: total_value = 300.

```
mysql> call calculate_total1();
+-------------+
| total_value |
+-------------+
|         300 |
+-------------+
```

Retrieve values using SELECT ... INTO

```
DELIMITER //
```

```
CREATE PROCEDURE get_student_marks(IN p_id INT)
BEGIN
    DECLARE v_marks DECIMAL(5,2);

    SELECT marks
    INTO v_marks
    FROM student
    WHERE student_id = p_id;

    SELECT v_marks AS marks;
END //

DELIMITER ;

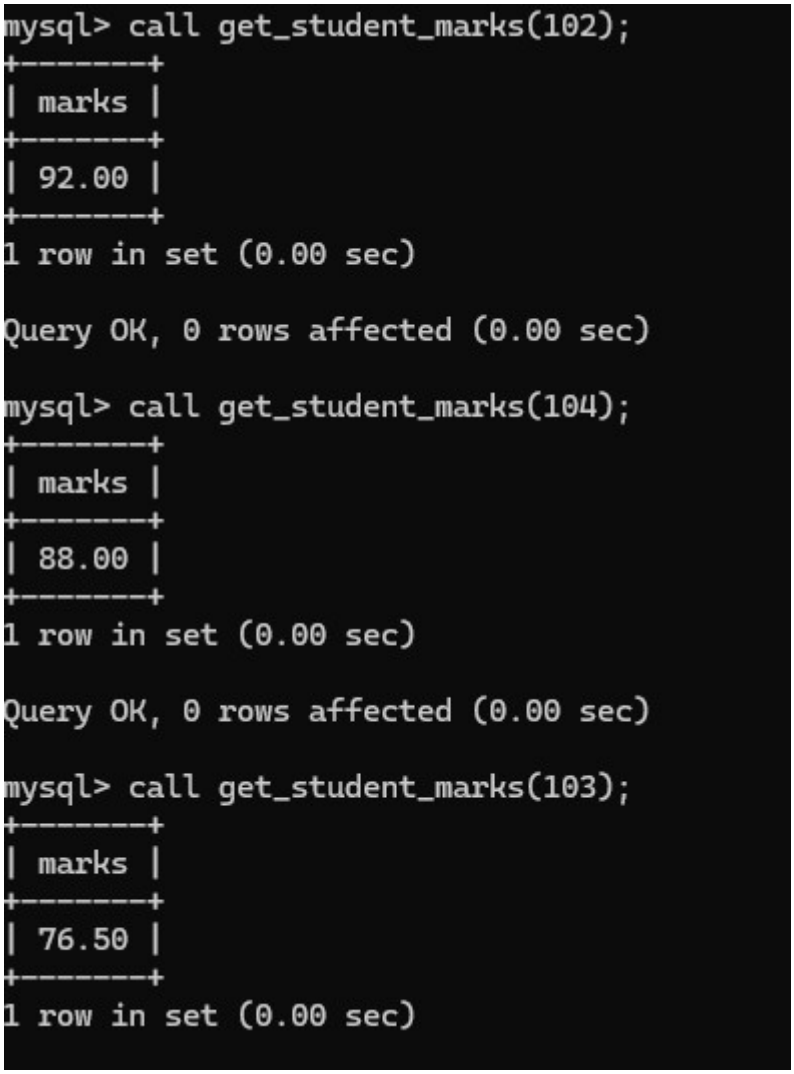
CALL get_student_marks(102);
```

```
mysql> delimiter $$
mysql>  create procedure get_student_marks(In s_id int)
    ->
    ->  begin
    ->
    ->  declare v_marks decimal(5,2);
    ->
    -> select marks
    -> into v_marks
    -> from student where student_id=s_id;
    ->
    -> select v_marks as marks;
    ->
    -> end $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> delimiter ;
```

The procedure receives 102 through p_id.
The query finds the marks for student 102.
INTO v_marks stores the selected value in the local variable.
The final SELECT displays the value.

SELECT ... INTO should return one row when assigning a single value. If no row is found, MySQL raises a no-data condition; without a suitable handler, the call reports an error. If multiple rows are returned, MySQL raises an error. A primary-key condition is a useful way to identify one row.



11. Local Variables vs User-Defined Variables

A user-defined variable begins with @ and is available across statements in the current MySQL session.

```
SET @total = 100;
SELECT @total;

SELECT COUNT(*)
INTO @student_count
FROM student;

SELECT @student_count;
```

```
mysql> delimiter //
mysql> create procedure variable_demo()
    -> begin
    -> declare x int default 20;
    ->
    -> select x as local_value,
    -> @x as session_value;
    -> end //
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> delimiter;
```

Feature	Local variable	User-defined variable
Declaration	DECLARE total INT;	No declaration required.
Name	total	@total
Scope	Declaring compound block	Current session
Initial value	NULL if no default is specified	NULL if not previously assigned
Typical use	Intermediate procedure logic	Temporary values shared by statements in a session
After procedure finishes	Not directly accessible outside its block	Available in the same session until changed or session ends

Demonstration

```
SET @x = 10;

DELIMITER //

CREATE PROCEDURE variable_demo()
BEGIN
    DECLARE x INT DEFAULT 20;

    SELECT x AS local_value,
           @x AS session_value;
END //

DELIMITER ;

CALL variable_demo();
SELECT @x;
```

Inside the procedure, local_value is 20 and session_value is 10. The final SELECT @x also returns 10. The names x and @x refer to different variables.

For routine calculations inside procedures, prefer local variables unless there is a specific reason to use a session variable.

Analogy: A local variable is like a value written in a working notebook for one exercise. A session variable is like a value written on a board that can be used by later exercises in the same session.

13. Procedure Parameters: IN, OUT and INOUT

Parameters allow a procedure to communicate with the caller.

Mode	Meaning	Example use
IN	Passes an input value into the procedure.	Pass a student ID to retrieve details.
OUT	Returns a value through a parameter.	Return the total number of students.
INOUT	Receives an initial value and returns its modified value.	Receive a number and return its doubled value.

IN parameter: retrieve a student

```
DELIMITER //

CREATE PROCEDURE get_student_by_id(IN p_id INT)
BEGIN
    SELECT *
    FROM student
    WHERE student_id = p_id;
END //

DELIMITER ;

CALL get_student_by_id(101);
```

The caller passes 101.
MySQL assigns the value to p_id.
The query uses p_id in the WHERE condition.
The matching record is returned as a result set.

OUT parameter: return a student count

```
DELIMITER //

CREATE PROCEDURE get_student_count(OUT p_count INT)
BEGIN
    SELECT COUNT(*)
    INTO p_count
    FROM student;
END //

DELIMITER ;
```

```
CALL get_student_count(@total);
SELECT @total AS total_students;
```

```
mysql> delimiter //
mysql>
mysql> create procedure get_stduent_count( out s_count int)
    -> begin
    -> select count(*)
    -> into s_count
    -> from student;
    ->
    -> end //
Query OK, 0 rows affected (0.01 sec)

mysql> delimiter ;
mysql>
mysql> call get_stduent_count(@total_student);
Query OK, 1 row affected (0.00 sec)

mysql> select @total_student;
+----------------+
| @total_student |
+----------------+
|              4 |
+----------------+
1 row in set (0.00 sec)
```

The procedure calculates the number of student records and places the result in p_count. The caller uses @total to receive the value and then displays it.

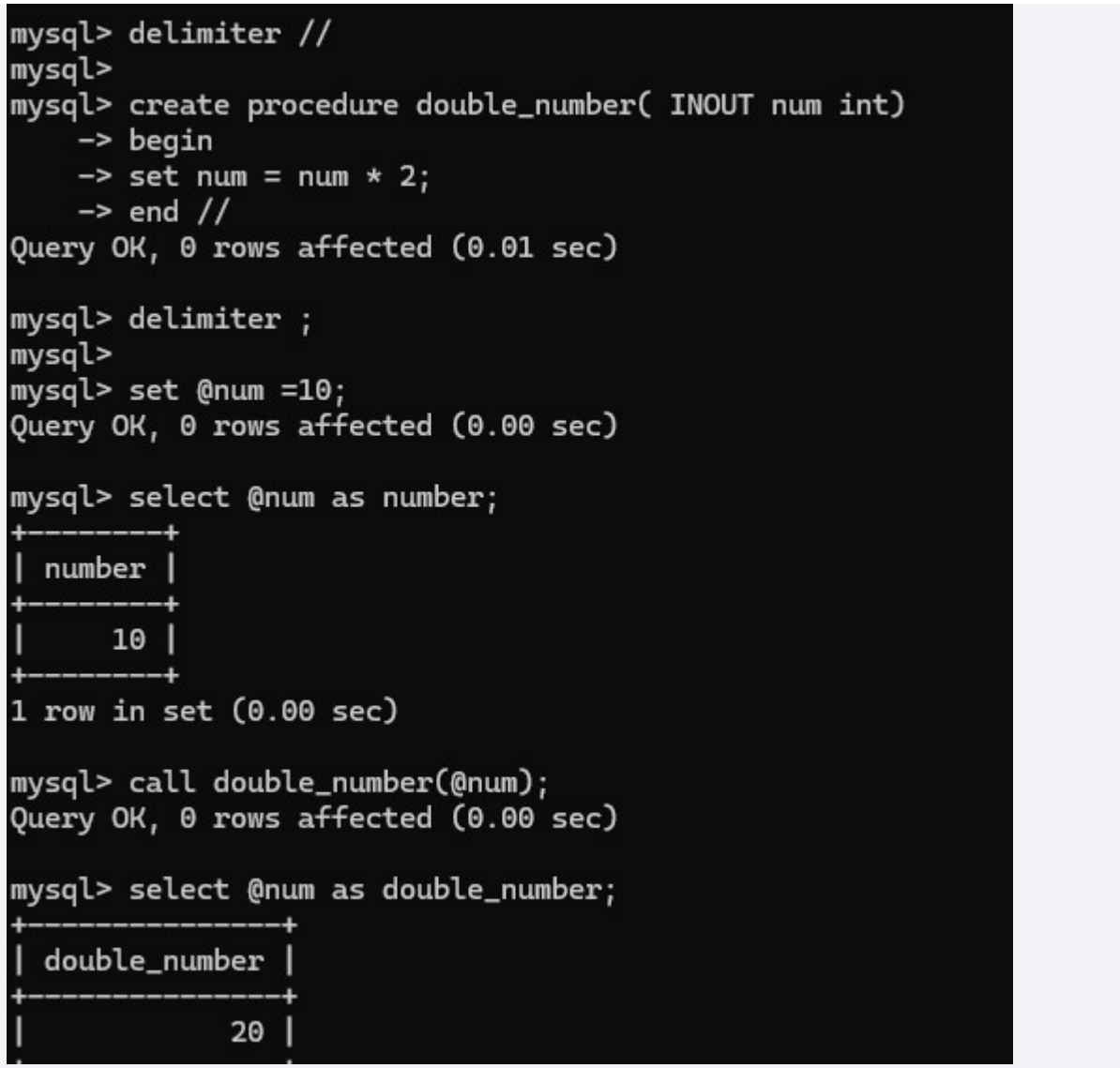
INOUT parameter: double a number

```
DELIMITER //

CREATE PROCEDURE double_number(INOUT p_num INT)
BEGIN
    SET p_num = p_num * 2;
END //

DELIMITER ;

SET @num = 10;
CALL double_number(@num);
SELECT @num AS doubled_value;
```



```
mysql> delimiter //
mysql>
mysql> create procedure double_number( INOUT num int)
    -> begin
    -> set num = num * 2;
    -> end //
Query OK, 0 rows affected (0.01 sec)

mysql> delimiter ;
mysql>
mysql> set @num =10;
Query OK, 0 rows affected (0.00 sec)

mysql> select @num as number;
+--------+
| number |
+--------+
|     10 |
+--------+
1 row in set (0.00 sec)

mysql> call double_number(@num);
Query OK, 0 rows affected (0.00 sec)

mysql> select @num as double_number;
+---------------+
| double_number |
+---------------+
|            20 |
```

The caller initializes @num to 10. The procedure doubles the value, and the caller receives 20.

A caller normally uses a user-defined variable such as @num for an INOUT argument, because it must supply an initial value and receive the modified value.

Parameter comparison

Feature	IN	OUT	INOUT
Pass value into procedure	Yes	Not used as normal input	Yes
Return value through parameter	No	Yes	Yes
Internal parameter can be assigned	Yes, but the change is not returned through the argument	Yes	Yes
Example	Student ID	Student count	Double a number

question: If a procedure receives a department ID and displays all students in that department, which parameter mode should be used?